Guía de exposición y demostración

Ruben Vigil · 4 de noviembre de 2026

Guion de apoyo para las 12 diapositivas. Ajustar la duración al tiempo indicado por el docente; no hay un tiempo de exposición impuesto por este documento.

Primera parte: problema, datos y metodología

1. Machine Learning para planificar entregas

Presentar el objetivo: usar datos reales para estimar duración y riesgo. La entrega, repartidores y efectivo son simulados; no es una operación comercial desplegada.

2. Qué problema resuelve el prototipo

Separar predicción de reglas. El modelo no calcula precios ni decide el repartidor. Esas decisiones se prueban con políticas explícitas y ejemplos, sin presentarlas como resultados de ML.

3. Arquitectura que ya funciona

Explicar por qué FastAPI y Spark no compiten: FastAPI recibe HTTP; scikit-learn predice; Pandas/Dask/PySpark procesan datos offline. MongoDB guarda operación; S3 guarda archivos. La API ya se ejecuta en ECS/Fargate y usa Atlas/S3 con roles IAM. Se verificaron predicción y persistencia; el acceso está restringido a la IP del presentador. Next.js sigue pendiente.

4. Datos observados y separación temporal

Distinguir intentos de pedidos. La purga evita entrenar con entregas que todavía no se conocían al cortar. El dataset es chino y de ventana corta; no representa mediciones de El Salvador.

5. Pandas, Dask y PySpark: resultados equivalentes

Los tiempos son resultados previos conservados, no estimaciones. Dask usó dos trabajadores y Spark se ejecutó localmente con dos hilos. Se separan arranque y pipeline. La normalización de CSV no demuestra identidad de escritores originales.

6. Qué aprenden los dos modelos actuales

La distancia aproximada hace que ETA responda al recorrido. El clasificador aprende la relación entre contexto, plazo y retraso observado. No usa tráfico, vehículo o tiempos de recogida posteriores para cotizar.

Guion: resultados y cierre

7. Resultado de duración: error menor

MAE significa error absoluto medio, no promesa de error máximo. El RF espacial tuvo un MAE de test apenas menor, pero no cambiamos al ganador mirando test. R^2 no equivale a porcentaje de exactitud.

8. Resultado de retraso: mejora modesta

La prevalencia de retraso es aproximadamente 15%, por eso acertar muchos pedidos puntuales no basta. El modelo supera la referencia constante, pero no debe venderse como garantía o probabilidad calibrada localmente.

9. También se entrenó una red neuronal pequeña

Mostrar que la parte académica compara diferentes algoritmos y una MLP real. El anexo contiene la curva de 30 épocas de validación y la elección de siete. No confundir este experimento temporal con la revisión geográfica.

10. Demo: predicción AWS y flujo local

Abrir el Swagger AWS indicado en la guía. Hay tres ejemplos con plazos 30/60/45 y datos ficticios en Atlas: 3 negocios, 2 repartidores y 3 pedidos solicitados. Comparar predicciones reales sin prometer monotonicidad. Las cifras de esta diapositiva son resultados previos con fecha fija. La demo completa de entrega/efectivo se ejecutó localmente, no en AWS. Si no hay conexión, identificar expresamente los recibos como evidencia guardada.

11. Límites que deben quedar claros

La aclaración oficial de Meituan permite distancias relativas con coordenadas desplazadas; la codificación 1e6 sigue siendo inferencia explícita. Explicar límites demuestra rigor y separa un prototipo académico de un servicio comercial.

12. Aporte alcanzado y siguiente validación

Cerrar con el aporte real, sin prometer producción. Las fuentes, detalles y evidencia se entregan en Word/PDF y ZIP. La API en AWS, modelos S3 y persistencia Atlas están comprobados. Distinguir ese alcance del flujo completo local. Next.js, evaluación local y validación bajo carga siguen pendientes.

Demostración: AWS y alternativa local

Antes de empezar

Abrir Swagger en http://100.54.11.78:8000/docs desde la conexión autorizada. Consultar el README si la IP cambió. Tener presentación, reportes y evidencias disponibles; no mostrar .env o credenciales.

1. Predicción en AWS

• Seleccionar POST /predict/delivery, elegir el ejemplo de recorrido corto y pulsar Try it out / Execute.

• Mostrar estimated_minutes, late_probability y model_source=s3. La probabilidad se expresa entre 0 y 1, no directamente como porcentaje.

• Elegir mismo recorrido con plazo 60 y luego el recorrido diferente con plazo 45. Son entradas ilustrativas; las respuestas las calculan los modelos reales.

• No usar un plazo de 1 minuto como ejemplo principal: queda fuera del rango observado (mínimo 30). Revisar outside_training_range.

2. Datos ficticios persistidos

Consultar GET /businesses, /couriers y /orders: 3 negocios, 2 repartidores y 3 pedidos demo-expo-. Son registros SIMULADOS, no dataset de entrenamiento; los pedidos permanecen solicitados. No se ejecutaron cobros o entregas reales.

3. Flujo completo de efectivo: alternativa local

La API cloud tiene predicción y CRUD verificados. El flujo completo de entrega/efectivo y su recibo se demostraron localmente; no presentarlo como una prueba cloud. Preparar Docker siguiendo README y ejecutar:

```
export DELIVERY_RUN_UID="$(id -u)" DELIVERY_RUN_GID="$(id -g)"
export DELIVERY_AWS_CONFIG_DIR="$HOME/.aws"
DELIVERY_DEMO_ENABLED=1 docker compose -f compose.yaml -f compose.s3.yaml up --build -d --wait
python3 scripts/demo_delivery.py
```

Resultados previos del ensayo local

Escenario ilustrativo	ETA registrada	Riesgo registrado
1.11 km / plazo 30	23.10 min	17.67%
5.56 km / plazo 30	45.38 min	70.60%
5.56 km / plazo 45	45.38 min	32.89%

Son salidas guardadas de escenarios con fecha fija, no métricas nuevas o resultados prefijados de Swagger. El CLI usa la hora actual y puede producir valores diferentes. El health antiguo describe v1; verificar v2 mediante /predict/delivery.

Preguntas frecuentes y plan de respaldo

¿Qué parte es Machine Learning?

Dos modelos predicen duración y probabilidad de retraso. Precios, elegibilidad de vehículos, rutas y registro de efectivo son reglas configurables.

¿Por qué FastAPI y no Spark?

FastAPI recibe HTTP y scikit-learn ejecuta los modelos. Spark se usó para demostrar procesamiento offline equivalente; no hace falta para una predicción pequeña por solicitud.

¿Qué calcula Haversine?

Una separación esférica aproximada entre coordenadas. No sigue calles ni estima tráfico. Es una fórmula usada para construir una entrada del modelo y aplicar reglas.

¿Por qué no se eligió el menor MAE de test?

Se seleccionó por validación temporal antes de evaluar test. Elegir después por el mejor test alteraría la función de ese conjunto de evaluación.

¿Esto ya funciona para El Salvador?

La demo funciona, pero las métricas provienen de China. Falta validación local. La escala de coordenadas es inferida y el riesgo tiene discriminación modesta.

¿Dónde están los pesos y la base?

Los pesos versionados están en S3 y se descargan/verifican al inicio; luego se usan en memoria. La API AWS usa Atlas. El flujo completo de efectivo también conserva una demo local en Docker.

¿Qué falta para operación pública?

La API ya funciona en Fargate con roles IAM, S3 y Atlas. Faltan Next.js, datos/evaluación locales, calibración, controles de aplicación, HTTPS y prueba de carga.

Si falla la demo o no hay conexión

Si AWS no responde, verificar la IP actual y su permiso /32 en Atlas. Evitar redesplegar durante la exposición: reemplazar tarea puede cambiar IP. La API descarga pesos solo al iniciar. No reiniciar innecesariamente. Ante un 503 o conexión ausente, mostrar el PDF y el recibo/evidencias guardados e identificarlos expresamente como resultados previos. No presentar una captura o salida guardada como si fuera una llamada nueva.

Para detener la demo local conservando sus datos: docker compose -f compose.yaml -f compose.s3.yaml down. Mantener las variables exportadas de la preparación; evitar --volumes si se desea conservar la demo.